COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010	**SECTION:**	CPE21S3
DATE PERFORMED:	July 23, 2026	**DATE SUBMITTED:**	July 28, 2026
NAME:	Carlo Mari B. Asajar	**INSTRUCTOR:**	Engr. Jimlord M. Quejado

ACTIVITY NO. 3.1 LINKED LISTS

1. PROCEDURE OUTPUT

ILO A: Construct C++ code for a singly and doubly linked list in C++

A.1. Singly Linked List

To start, we will do a simple implementation of a linked list. We must keep in mind the visual representation of the individual nodes that will make up our linked list:

Every node in a singly linked list will have 2 compartments, the data and the pointer to the next. The data contains the element of a single type that we want it to contain. The link will point to the next node in our list.

Implementation will follow the given steps:

1. Create the node pointers and initialize as NULL
2. Create new instances of the node class and allocate them in the heap
3. Define every node in the list
4. Point the last node to null

Although we have created the linked list, this is an implementation that is useless and meant only to show what we want to happen for the given output.

Run your code and screenshot the output, then briefly discuss how the output came to be and how could it be improved?
(Table 3-1. Output of Initial/Simple Implementation)

Screenshot	**My Code: (simple_singly_LL.h)**

ILOA > simple_singly_LL.h > SinglyList<T>

```h
#ifndef SIMPLE_SINGLY_LL_H
#define SIMPLE_SINGLY_LL_H

//TEMPLATING
template <typename T>
class SinglyList{
    public:
        T data;
        SinglyList<T> *next = nullptr;

};

#endif
```

My Code: (simple_main.cpp)

ILOA > simple_main.cpp > main()

```cpp
#include <iostream>
#include "simple_singly_LL.h"

int main(){

    // Create and allocate memory for each node
    SinglyList<char>* head = new SinglyList<char>;
    SinglyList<char>* second = new SinglyList<char>;
    SinglyList<char>* third = new SinglyList<char>;
    SinglyList<char>* fourth = new SinglyList<char>;
    SinglyList<char>* fifth = new SinglyList<char>;

    // Assign values and link the nodes together
    head->data = 'C';
    head->next = second;

```

```
17      second->data = 'A';
18      second->next = third;
19
20      third->data = 'R';
21      third->next = fourth;
22
23      fourth->data = 'L';
24      fourth->next = fifth;
25
26      fifth->data = 'O';
27      fifth->next = nullptr;
28
29      std::cout << head->data << second->data << std::endl;
30      std::cout << head->next << second->next->data << std::endl;
31  }
```

My Output:

```
CA
0x77c1b0R
PS C:\Users\carlo\Downloads\ASAJAR_CPE 0
10_Hands-on Activity 3.1_Linked Lists>
```

Discussion

The code shows how a simple singly linked list can be created by making several nodes and connecting them using pointers. The important code started with ***template <typename T>*** and the ***SinglyList*** class, where ***T*** data was used to store the value and ***SinglyList<T> *next = nullptr*** was used to point to the next node. In the ***main()*** function, the code used new ***SinglyList<char>*** to create and allocate the five nodes named ***head, second, third, fourth,*** and **fifth** in the heap. The values were then assigned ***using head->data = 'C', second->data = 'A', third->data = 'R', fourth->data = 'L'***, and ***fifth->data = 'O'***, which made the nodes contain the letters ***C, A, R, L,*** and ***O***. The important pointer codes such as ***head->next = second, second->next = third, third->next = fourth***, and ***fourth->next = fifth*** connected each node to the next one, while ***fifth->next = nullptr*** showed that the last node had no next node.

	The output code ***std::cout << head->data << second->data << std::endl;*** displayed the first two data values, while ***std::cout << head->next << second->next->data << std::endl;*** showed the pointer address of the next node and the data connected after it. It was a little confusing at first because ***head->next*** does not show the actual letter but instead gives a memory address, but it helped show that the pointer was really connected to another node.

Table 3-1. Output of Initial/Simple Implementation

Imagine if we had to make multiple items in the list in the hundreds or thousands! This would be too tedious, ineffective, and inefficient. So, we must implement certain methods. These methods/operations associated with the linked lists are:

- Traversal
- Insertion at head
- Insertion at any part of the list
- Insertion at the end
- Deletion of a node

Linked List Traversal

Sometimes we have a linked list, and we need to insert a node somewhere other than at the end of the list. We will look at a couple of different ways to insert a node into an existing list.

To insert a node at the head:

1. Allocate memory for the new node
2. Put our data into the new node
3. Set Next of the new node to point to the previous Head
4. Reset Head to point to the new node

To insert a node at any location between the head and tail:

1. Check if it is the head node (previous node is null)
2. If null, print "Previous node cannot be null."
3. Allocate a new node
4. Store data in the new node
5. Point new node to the node previous node was pointing to
6. Point previous node to the new node

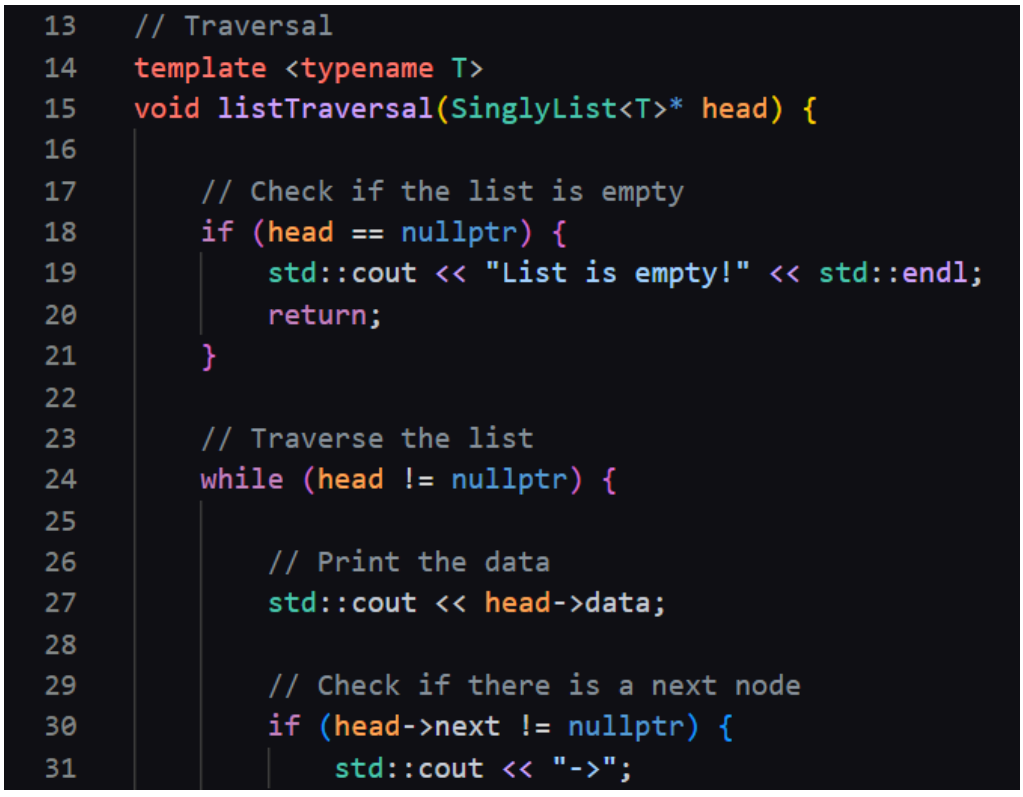
To insert a node at the end:

1. Allocate new node
2. Dereference to the head node
3. Store data in new node
4. Point next of new node to NULL
5. Traverse the list until next of the node is null
6. Point the next of the current node to the new node

To delete a node from linked list:

1. Find the previous node of the node to be deleted.
2. Change the next of the previous nodes.
3. Free memory for the node to be deleted.

Create code for each of the pseudocode given for all list operations above. Provide screenshots.
(Table 3-2. Code for the List Operations)

Operation	Screenshot
Traversal	

```
13  // Traversal
14  template <typename T>
15  void listTraversal(SinglyList<T>* head) {
16
17      // Check if the list is empty
18      if (head == nullptr) {
19          std::cout << "List is empty!" << std::endl;
20          return;
21      }
22
23      // Traverse the list
24      while (head != nullptr) {
25
26          // Print the data
27          std::cout << head->data;
28
29          // Check if there is a next node
30          if (head->next != nullptr) {
31              std::cout << "->";
```

```
32          }
33
34          // Move to the next node
35          head = head->next;
36      }
37  }
```

Insertion at head

```
39  // Node Insertion (Head)
40  template <typename T>
41  void sllinsertHead(T newData, SinglyList<T>** currentHead) {
42
43      // Create a new node
44      SinglyList<T>* newNode = new SinglyList<T>;
45
46      // Store data in the new node
47      newNode->data = newData;
48
49      // Connect the new node to the current head
50      newNode->next = *currentHead;
51
52      // Make the new node the head
53      *currentHead = newNode;
54  }
```

Insertion at any part of the list

```
56  // Node Insertion (Body)
57  template <typename T>
58  void sllGeneralInsert(T newData, SinglyList<T>* prevNode) {
59
60      // Check if the previous node is null
61      if (prevNode == nullptr) {
62          std::cout << "Previous node cannot be null!" << std::endl;
63          return;
64      }
65
66      // Create a new node
67      SinglyList<T>* newNode = new SinglyList<T>;
68
69      // Store data in the new node
70      newNode->data = newData;
71
72      // Point the new node to the node after the previous node
73      newNode->next = prevNode->next;
```

```
74
75      // Connect the previous node to the new node
76      prevNode->next = newNode;
77  }
```

Insertion at the end

```
79  // Node Insertion (End)
80  template <typename T>
81  void sllinsertEnd(T newData, SinglyList<T>** head) {
82
83      // Create a new node
84      SinglyList<T>* newNode = new SinglyList<T>;
85
86      // Store data in the new node
87      newNode->data = newData;
88
89      // Point next of the new node to NULL
90      newNode->next = nullptr;
91
92      // If the list is empty
93      if (*head == nullptr) {
94          *head = newNode;
95          return;
96      }
97
98      // Traverse to the last node
99      SinglyList<T>* currentNode = *head;
100
101     while (currentNode->next != nullptr) {
102         currentNode = currentNode->next;
103     }
104
105     // Connect the last node to the new node
106     currentNode->next = newNode;
107 }
```

Deletion of a node

```
109  // Delete Nodes
110  template <typename T>
111  void sllDeleteNodes(T findData, SinglyList<T>** head) {
112
113      if (*head == nullptr) return;
114
115      SinglyList<T>* currNode = *head;
116      SinglyList<T>* prevNode = nullptr;
117
118      // Search for the node
119      while (currNode != nullptr && currNode->data != findData) {
120          prevNode = currNode;
121          currNode = currNode->next;
122      }
123
124      // Data not found
125      if (currNode == nullptr) return;
126
127      // If the node is the head
128      if (prevNode == nullptr) {
129          *head = currNode->next;
130      }
131      else {
132          prevNode->next = currNode->next;
133      }
134
135      // Delete the found node
136      delete currNode;
137  }
```

Deletion of the list

```
139  // Delete All Nodes
140  template <typename T>
141  void sllDeleteList(SinglyList<T>** head) {
142
143      SinglyList<T>* current = *head;
144
145      while (current != nullptr) {
146
147          SinglyList<T>* temp = current;
```

```
148         current = current->next;
149
150         // Delete the current node
151         delete temp;
152     }
153
154     // Set the head to null
155     *head = nullptr;
156 }
```

Table 3-2. Code for the List Operations

In your driver function, show the use of each list operation and show the output with a descriptive caption for each.
(Table 3-3. Code and Analysis for Singly Linked Lists)

a.	**Source Code**	**(singly_LL.h)**
		
		`// Traversal`
		`template <typename T>`
		`void listTraversal(SinglyList<T>* head) {`
		
		`    // Check if the list is empty`
		`    if (head == nullptr) {`
		`        std::cout << "List is empty!" << std::endl;`
		`        return;`
		`    }`
		
		`    // Traverse the list`
		`    while (head != nullptr) {`
		
		`        // Print the data`
		`        std::cout << head->data;`

```
29          // Check if there is a next node
30          if (head->next != nullptr) {
31              std::cout << "->";
32          }
33
34          // Move to the next node
35          head = head->next;
36      }
37  }
```

(main_ILOA.1.cpp)

```
40      // Traverse and display the complete list
41      std::cout << "Traversal:" << std::endl;
42      listTraversal(head);
43      std::cout << "\n---------------------------------------\n" << std::endl;
```

Console

```
Traversal:
C->A->R->L->O
-----------------------------------------
```

b. **Source Code**

(singly_LL.h)

```
39  // Node Insertion (Head)
40  template <typename T>
41  void sllinsertHead(T newData, SinglyList<T>** currentHead) {
42
43      // Create a new node
44      SinglyList<T>* newNode = new SinglyList<T>;
45
46      // Store data in the new node
47      newNode->data = newData;
48
```

```
49      // Connect the new node to the current head
50      newNode->next = *currentHead;
51
52      // Make the new node the head
53      *currentHead = newNode;
54  }
```

(main_ILOA.1.cpp)

```
45      // Insert a new node at the beginning of the list
46      std::cout << "Insert a New Node at the Head:" << std::endl;
47      sllinsertHead('S', &head);
48      listTraversal(head);
49      std::cout << "\n--------------------------------------\n" << std::endl;
```

Console

```
Insert a New Node at the Head:
S->C->A->R->L->O
----------------------------------------
```

c. **Source Code**

(singly_LL.h)

```
56  // Node Insertion (Body)
57  template <typename T>
58  void sllGeneralInsert(T newData, SinglyList<T>* prevNode) {
59
60      // Check if the previous node is null
61      if (prevNode == nullptr) {
62          std::cout << "Previous node cannot be null!" << std::endl;
63          return;
64      }
65
66      // Create a new node
67      SinglyList<T>* newNode = new SinglyList<T>;
68
69      // Store data in the new node
70      newNode->data = newData;
```

```
71
72      // Point the new node to the node after the previous node
73      newNode->next = prevNode->next;
74
75      // Connect the previous node to the new node
76      prevNode->next = newNode;
77  }
```

(main_ILOA.1.cpp)

```
51      // Insert a new node after the third node
52      std::cout << "Insert a New Node after the Third Node:" << std::endl;
53      sllGeneralInsert('I', head->next->next);
54      listTraversal(head);
55      std::cout << "\n---------------------------------------\n" << std::endl;
```

Console

```
Insert a New Node after the Third Node:
S->C->A->I->R->L->O
---------------------------------------
```

d. **Source Code**

(singly_LL.h)

```
79  // Node Insertion (End)
80  template <typename T>
81  void sllinsertEnd(T newData, SinglyList<T>** head) {
82
83      // Create a new node
84      SinglyList<T>* newNode = new SinglyList<T>;
85
86      // Store data in the new node
87      newNode->data = newData;
88
89      // Point next of the new node to NULL
90      newNode->next = nullptr;
91
```

```
92      // If the list is empty
93      if (*head == nullptr) {
94          *head = newNode;
95          return;
96      }
97
98      // Traverse to the last node
99      SinglyList<T>* currentNode = *head;
100
101     while (currentNode->next != nullptr) {
102         currentNode = currentNode->next;
103     }
104
105     // Connect the last node to the new node
106     currentNode->next = newNode;
107 }
```

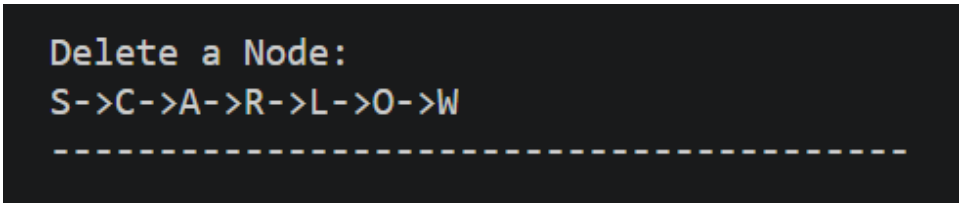
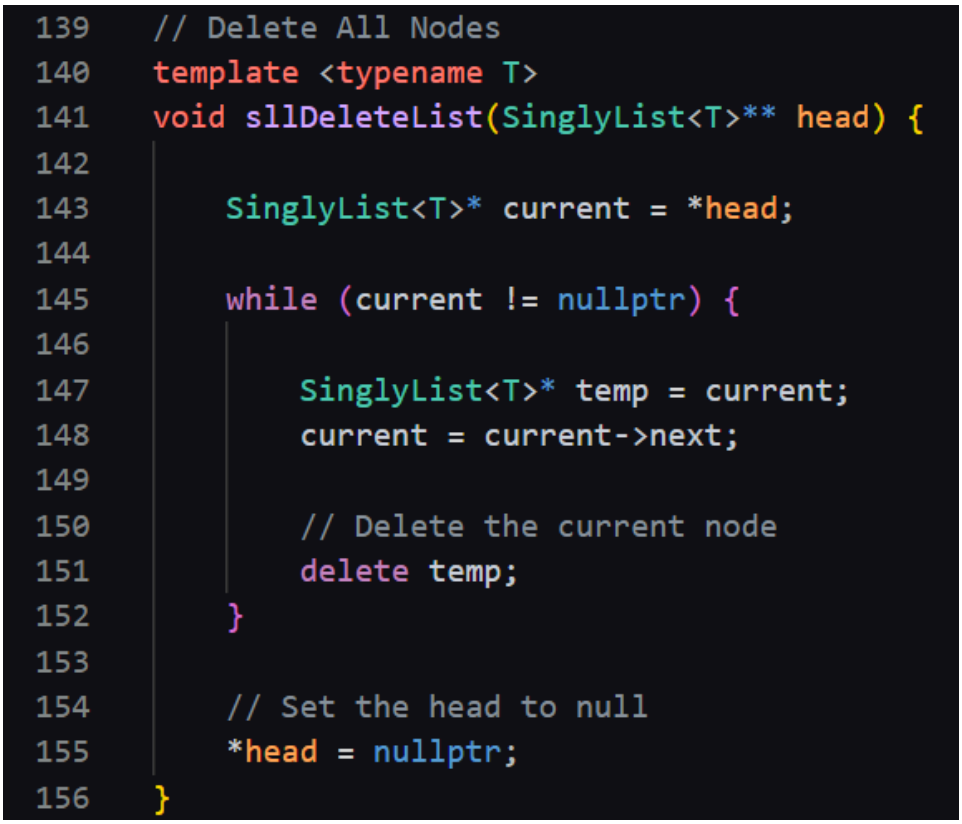
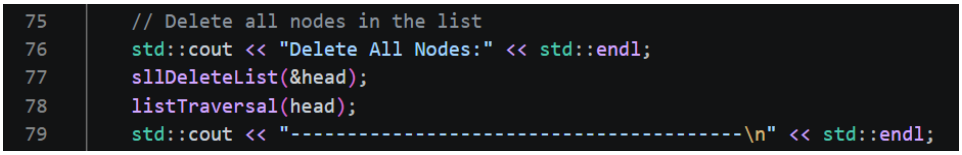
(main_ILOA.1.cpp)

```
57      // Insert a new node at the end of the list
58      std::cout << "Insert a New Node at the End:" << std::endl;
59      sllinsertEnd('W', &head);
60      listTraversal(head);
61      std::cout << "\n----------------------------------------\n" << std::endl;
```

Console

```
Insert a New Node at the End:
S->C->A->I->R->L->O->W
----------------------------------------
```

e.	**Source Code**	**(singly_LL.h)**
		
		`// Delete Nodes`
		`template <typename T>`
		`void sllDeleteNodes(T findData, SinglyList<T>** head) {`
		
		`    if (*head == nullptr) return;`
		
		`    SinglyList<T>* currNode = *head;`
		`    SinglyList<T>* prevNode = nullptr;`
		
		`    // Search for the node`
		`    while (currNode != nullptr && currNode->data != findData) {`
		`        prevNode = currNode;`
		`        currNode = currNode->next;`
		`    }`
		
		`    // Data not found`
		`    if (currNode == nullptr) return;`
		
		`    // If the node is the head`
		`    if (prevNode == nullptr) {`
		`        *head = currNode->next;`
		`    }`
		`    else {`
		`        prevNode->next = currNode->next;`
		`    }`
		
		`    // Delete the found node`
		`    delete currNode;`
		`}`
		
		(main_ILOA.1.cpp)
		
		`    // Remove the node containing the specified value`
		`    std::cout << "Delete a Node:" << std::endl;`
		`    sllDeleteNodes('I', &head);`
		`    listTraversal(head);`
		`    std::cout << "\n----------------------------------------\n" << std::endl;`

	Console	
f.	Source Code	(singly_LL.h)  (main_ILOA.1.cpp) 


```
139   // Delete All Nodes
140   template <typename T>
141   void sllDeleteList(SinglyList<T>** head) {
142
143       SinglyList<T>* current = *head;
144
145       while (current != nullptr) {
146
147           SinglyList<T>* temp = current;
148           current = current->next;
149
150           // Delete the current node
151           delete temp;
152       }
153
154       // Set the head to null
155       *head = nullptr;
156   }
```

```
75      // Delete all nodes in the list
76      std::cout << "Delete All Nodes:" << std::endl;
77      sllDeleteList(&head);
78      listTraversal(head);
79      std::cout << "----------\n" << std::endl;
```

	Console	

Table 3-3. Code and Analysis for Singly Linked Lists

A.2. Doubly Linked List

The singly linked list allows for direct access from a list node only to the next node in the list. A doubly linked list allows convenient access from a list node to the next node and also to the preceding node on the list.

The doubly linked list node accomplishes this in the obvious way by storing two pointers: one to the node following it (as in the singly linked list), and a second pointer to the node preceding it.

This means that in addition to our implementation of the singly linked list, we'll have an additional compartment.

Modify the given operations used in the singly linked lists to work on the new construct of a doubly linked list. Provide a screenshot of your code and analysis.
(Table 3-4. Modified Operations for Doubly Linked Lists)

Screenshots(s)	Analysis
ILOA > C doubly_LL.h > ... 1 #ifndef DOUBLE_LL_H 2 #define DOUBLE_LL_H 3 4 #include <iostream> 5 6 // Creating a template 7 template <typename T> 8 class DoubleList {	The doubly linked list starts by modifying the node structure from the singly linked list. The important code is ***DoubleList<T>* prev = nullptr***; and ***DoubleList<T>* next = nullptr;***, where ***prev*** connects the node to the previous node while ***next*** connects it to the next node. The ***T data*** is still used to store the value inside the node. The ***#ifndef***

```
 9    public:
10        T data;
11        DoubleList<T>* prev = nullptr;
12        DoubleList<T>* next = nullptr;
13    };
```

DOUBLE_LL_H, ***#define DOUBLE_LL_H***, ***#include <iostream>***, and ***#endif*** are also included as part of the header file. Because of the added ***prev*** pointer, the list can now move in two directions instead of only moving forward.

```
15  // Create a new node
16  template <typename T>
17  DoubleList<T>* createNode(T newData) {
18
19      // Dynamically allocate memory for the new node
20      DoubleList<T>* newNode = new DoubleList<T>;
21
22      newNode->data = newData;
23      newNode->prev = nullptr;
24      newNode->next = nullptr;
25
26      // Return the address of the newly created node
27      return newNode;
28  }
```

The ***createNode()*** function is used to make a new node and allocate memory for it. The important code is ***DoubleList<T>* newNode = new DoubleList<T>;***, which creates the node in the heap, while ***newNode->data = newData;***, ***newNode->prev = nullptr;***, and ***newNode->next = nullptr;*** set the starting values. This makes sure that the new node has its data and both pointers properly initialized before it is connected to the list.

```
30  // Traverse the list
31  template <typename T>
32  void dllTraverse(DoubleList<T>* currentNode) {
33
34      DoubleList<T>* tail = nullptr;
35
36      // Check if the list is empty
37      if (currentNode == nullptr) {
38          std::cout << "The list is empty." << std::endl;
39          return;
40      }
41
42      // Forward traversing
43      std::cout << "Forward: " << std::endl;
44
45      while (currentNode != nullptr) {
46
47          std::cout << currentNode->data << " ";
48
49          // Save the current node as the last visited node
50          tail = currentNode;
```

The ***dllTraverse()*** function was modified so the list can be viewed in both forward and backward directions. The important code for moving forward is ***currentNode = currentNode->next;***, while ***tail = tail->prev;*** is used to move backward. During the first loop, the last visited node is saved in the ***tail***, so it can later be used to travel back through the list. This shows one of the main differences of the doubly linked list because the nodes can be accessed from both sides.

```
51
52          // Move to the next node
53          currentNode = currentNode->next;
54      }
55
56      std::cout << std::endl;
57
58      // Backward traversing
59      std::cout << "Backward: " << std::endl;
60
61      while (tail != nullptr) {
62
63          std::cout << tail->data << " ";
64
65          // Move toward the previous node
66          tail = tail->prev;
67      }
68
69      std::cout << std::endl;
70  }
```

```
72  // Insert at the head
73  template <typename T>
74  void dllInsertHead(T newData, DoubleList<T>*& head) {
75
76      // Create a new node
77      DoubleList<T>* newNode = createNode(newData);
78
79      // If the list is empty
80      if (head == nullptr) {
81          head = newNode;
82          return;
83      }
84
85      // Connect the new node to the current head
86      newNode->next = head;
87      head->prev = newNode;
88
89      // Make the new node the head
90      head = newNode;
91  }
```

The ***dllInsertHead()*** function adds a new node at the beginning of the list. The important code is ***newNode->next = head;***, which connects the new node to the old head, and ***head->prev = newNode;***, which connects the old head back to the new node. The code ***head = newNode;*** then makes the new node the new first node. This is different from the singly linked list because the previous connection also needs to be updated.

```cpp
// Insert between two nodes
template <typename T>
void dllInsertAny(
    T newData,
    DoubleList<T>* prevNode,
    DoubleList<T>* currentNode
) {

    // Check if the previous node is null
    if (prevNode == nullptr) {
        std::cout << "Previous node cannot be null." << std::endl;
        return;
    }

    // Create a new node
    DoubleList<T>* newNode = createNode(newData);

    // Connect the new node
    newNode->next = currentNode;
    newNode->prev = prevNode;

    // Connect the previous node to the new node
    prevNode->next = newNode;

    // Connect the current node back to the new node
    if (currentNode != nullptr) {
        currentNode->prev = newNode;
    }
}
```

The ***dllInsertAny()*** function is used to place a new node between two existing nodes. The important code is ***newNode->next = currentNode;*** and ***newNode->prev = prevNode;***, which connects the new node to both sides. The code ***prevNode->next = newNode;*** connects the previous node forward, while ***currentNode->prev = newNode;*** connects the next node backward. Because both connections are updated, the list stays properly connected after the insertion.

```cpp
// Insert at the end
template <typename T>
void dllInsertEnd(T newData, DoubleList<T>** head) {

    // Create a new node
    DoubleList<T>* newNode = createNode(newData);

    // If the list is empty
    if (*head == nullptr) {
        *head = newNode;
        return;
    }

    // Traverse to the last node
    DoubleList<T>* currentNode = *head;

    while (currentNode->next != nullptr) {
        currentNode = currentNode->next;
    }

    // Connect the last node to the new node
    currentNode->next = newNode;
    newNode->prev = currentNode;
}
```

The ***dllInsertEnd()*** function adds a new node after the last node in the list. The important code is ***currentNode->next = newNode;*** and ***newNode->prev = currentNode;***. The first code connects the old last node to the new node, while the second code gives the new node a connection back to the old last node. The ***while (currentNode->next != nullptr)*** loop is also important because it finds the last node before the new node is added.

```
148  // Delete a node
149  template <typename T>
150  void dllDeleteNode(
151      DoubleList<T>* nodeToDelete,
152      DoubleList<T>** head
153  ) {
154
155      if (nodeToDelete == nullptr) {
156          return;
157      }
158
159      // Get previous and next nodes
160      DoubleList<T>* prevNode = nodeToDelete->prev;
161      DoubleList<T>* nextNode = nodeToDelete->next;
162
163      // Update the previous node
164      if (prevNode != nullptr) {
165          prevNode->next = nextNode;
166      }
167      else {
168          // Deleting the head node
169          *head = nextNode;
170      }
171
172      // Update the next node
173      if (nextNode != nullptr) {
174          nextNode->prev = prevNode;
175      }
176
177      // Delete the node
178      delete nodeToDelete;
179  }
```

The ***dllDeleteNode()*** function removes a specific node while keeping the remaining nodes connected. The important code is ***DoubleList<T>* prevNode = nodeToDelete->prev;*** and ***DoubleList<T>* nextNode = nodeToDelete->next;***, which saves the nodes on both sides. The code ***prevNode->next = nextNode;*** reconnects the previous node to the next node, while ***nextNode->prev = prevNode;*** reconnects the next node back to the previous one. Finally, ***delete nodeToDelete;*** removes the selected node from memory.

```
181  // Delete all nodes
182  template <typename T>
183  void dllDeleteList(DoubleList<T>*& head) {
184
185      DoubleList<T>* current = head;
186
187      while (current != nullptr) {
188
189          DoubleList<T>* temp = current;
```

The ***dllDeleteList()*** function removes all nodes from the doubly linked list. The important code is ***current = current->next;***, which moves through the list, and ***delete temp;***, which removes each node from memory. After all nodes are deleted, ***head = nullptr;*** makes the list empty.

```
190             current = current->next;
191
192             // Delete the current node
193             delete temp;
194         }
195
196         // Set the head to null
197         head = nullptr;
198     }
```

This operation is useful because it clears the whole list instead of deleting only one node at a time.

Table 3-4. Modified Operations for Doubly Linked Lists

My Code: (main_ILOA.2.cpp)

ILOA > main_ILOA.2.cpp > main()

```cpp
#include <iostream>
#include "doubly_LL.h"

int main() {

    // Creating the first nodes 'C', 'P', 'E'
    DoubleList<char>* head = createNode('C');
    DoubleList<char>* second = createNode('P');
    DoubleList<char>* third = createNode('E');

    // Link the nodes together
    head->next = second;
    second->prev = head;

    second->next = third;
    third->prev = second;

    // Line Divider for formatting
    std::cout << "\n---------------------------------------" << std::endl;

    // Traversal
    std::cout << "Traversal Testing:" << std::endl;
    dllTraverse(head);
    std::cout << "\n---------------------------------------" << std::endl;

```

```
26      // Insertion at the head
27      std::cout << "Insert Head Testing:" << std::endl;
28      dllInsertHead('X', head);
29      dllTraverse(head);
30      std::cout << "\n----------------------------------------" << std::endl;
31
32      // Insertion at any part of the list
33      std::cout << "Insert Any Node Testing:" << std::endl;
34
35      // First option
36      dllInsertAny('Y', head, head->next);
37      dllTraverse(head);
38
39      // Second option
40      DoubleList<char>* current = head;
41
42      while (current != nullptr && current->data != 'P') {
43          current = current->next;
44      }
45
46      if (current != nullptr) {
47          dllInsertAny('Y', current, current->next);
48      }
49
50      dllTraverse(head);
51      std::cout << "\n----------------------------------------" << std::endl;
52
53      // Insertion at the end
54      std::cout << "Insert End Testing:" << std::endl;
55      dllInsertEnd('Z', &head);
```

```
56      dllTraverse(head);
57      std::cout << "\n----------------------------------------" << std::endl;
58
59      // Deletion of a node
60      std::cout << "Delete Node Testing:" << std::endl;
61      dllDeleteNode(second, &head);
62      dllTraverse(head);
63      std::cout << "\n----------------------------------------" << std::endl;
64
65      // Delete all nodes
66      std::cout << "Delete All Nodes Testing:" << std::endl;
67      dllDeleteList(head);
```

```
68      dllTraverse(head);
69      std::cout << std::endl;
70
71      return 0;
72  }
```

My Output:

Part 1

```
----------------------------------------
Traversal Testing:
Forward:
C P E
Backward:
E P C

----------------------------------------
Insert Head Testing:
Forward:
X C P E
Backward:
E P C X

----------------------------------------
Insert Any Node Testing:
Forward:
X Y C P E
Backward:
E P C Y X
Forward:
X Y C P Y E
Backward:
E Y P C Y X
```

Part 2

```
----------------------------------------
Insert End Testing:
Forward:
X Y C P Y E Z
Backward:
Z E Y P C Y X

----------------------------------------
Delete Node Testing:
Forward:
X Y C Y E Z
Backward:
Z E Y C Y X

----------------------------------------
Delete All Nodes Testing:
The list is empty.

PS C:\Users\carlo\Downloads\ASAJAR_CPE 010_
Hands-on Activity 3.1_Linked Lists>
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO B: Solve given problems utilizing linked lists in C++

Problem Title: Implementing a Song Playlist using Linked List
Source: Packt Publishing

Problem Description:

In this activity, we'll look at some applications for which a singly linked list is not enough or not convenient. We will build a tweaked version that fits the application. We often encounter cases where we have to customize default implementations, such as when looping songs in a music player or in games where multiple players take a turn one by one in a circle.

These applications have one common property – we traverse the elements of the sequence in a circular fashion. Thus, the node after the last node will be the first node while traversing the list. This is called a circular linked list.

We'll take the use case of a music player. It should have following functions supported:

- Create a playlist using multiple songs.
- Add songs to the playlist.
- Remove a song from the playlist.
- Play songs in a loop (for this activity, we will print all the songs once).

Here are the steps to solve the problem:

- Design the basic structure that supports circular data representation.
- After that, implement the insert and delete functions in the structure.
- Implement a function for traversing the playlist.

The driver function should allow for common operations on a playlist such as: next, previous, play all songs, insert and remove.

My Code: (playlist_ILOB.h)

ILOB > C playlist_ILOB.h > ...

```
#ifndef PLAYLIST_H
#define PLAYLIST_H

#include <iostream>
#include <string>

// Node structure for the circular linked list
template <typename T>
class PlaylistNode {
public:
    T data;
    PlaylistNode<T>* next = nullptr;
};

// Create a new node
template <typename T>
PlaylistNode<T>* createNode(T newData) {
    PlaylistNode<T>* newNode = new PlaylistNode<T>;
    newNode->data = newData;
    newNode->next = nullptr;

    return newNode;
}

// Create a playlist using multiple songs
template <typename T>
PlaylistNode<T>* createPlaylist(T songs[], int size) {
    if (songs == nullptr || size <= 0) {
        return nullptr;
    }

    PlaylistNode<T>* head = nullptr;
    PlaylistNode<T>* tail = nullptr;

    for (int i = 0; i < size; i++) {
        PlaylistNode<T>* newNode = createNode(songs[i]);

        if (head == nullptr) {
            head = newNode;
```

```
40              tail = newNode;
41          }
42          else {
43              tail->next = newNode;
44              tail = newNode;
45          }
46      }
47
48      // Connect the last node back to the first node
49      tail->next = head;
50
51      return head;
52  }
53
54  // Add a song to the playlist
55  template <typename T>
56  void addSong(T newData, PlaylistNode<T>** head) {
57      PlaylistNode<T>* newNode = createNode(newData);
58
59      // If the playlist is empty
60      if (*head == nullptr) {
61          *head = newNode;
62          newNode->next = newNode;
63          return;
64      }
65
66      // Find the last node
67      PlaylistNode<T>* tail = *head;
68
```

```
69      while (tail->next != *head) {
70          tail = tail->next;
71      }
72
73      // Add the new song at the end
74      tail->next = newNode;
75      newNode->next = *head;
76  }
77
78  // Remove a song from the playlist
79  template <typename T>
```

```cpp
void removeSong(T findData, PlaylistNode<T>** head) {
    if (*head == nullptr) {
        std::cout << "The playlist is empty." << std::endl;
        return;
    }

    PlaylistNode<T>* current = *head;
    PlaylistNode<T>* previous = nullptr;

    // Search for the song
    do {
        if (current->data == findData) {
            break;
        }

        previous = current;
        current = current->next;

    } while (current != *head);

    // Song was not found
    if (current->data != findData) {
        std::cout << "Song not found." << std::endl;
        return;
    }

    // If there is only one song
    if (current->next == current) {
        delete current;
        *head = nullptr;
        return;
    }

    // If removing the first song
    if (current == *head) {
        PlaylistNode<T>* tail = *head;

        while (tail->next != *head) {
            tail = tail->next;
        }

```

```
121             *head = current->next;
122             tail->next = *head;
123 
124             delete current;
125             return;
126         }
127 
128         // Removing a song other than the first
129         previous->next = current->next;
130 
131         delete current;
132     }
133 
134     // Play all songs in the playlist
135     template <typename T>
136     void playAllSongs(PlaylistNode<T>* head) {
137         if (head == nullptr) {
138             std::cout << "The playlist is empty." << std::endl;
139             return;
140         }
141 
142         PlaylistNode<T>* current = head;
143 
144         do {
145             std::cout << current->data << std::endl;
146             current = current->next;
147 
148         } while (current != head);
149     }
150 
```

```
151     // Move to the next song
152     template <typename T>
153     void nextSong(PlaylistNode<T>*& current) {
154         if (current == nullptr) {
155             std::cout << "The playlist is empty." << std::endl;
156             return;
157         }
158 
159         current = current->next;
160 
161         std::cout << "Now Playing: " << current->data << std::endl;
162     }
```

```cpp

// Move to the previous song
template <typename T>
void previousSong(PlaylistNode<T>* head,
                  PlaylistNode<T>*& current) {

    if (head == nullptr || current == nullptr) {
        std::cout << "The playlist is empty." << std::endl;
        return;
    }

    PlaylistNode<T>* previous = head;

    // Find the node before the current node
    while (previous->next != current) {
        previous = previous->next;
    }

    current = previous;

    std::cout << "Now Playing: " << current->data << std::endl;
}

#endif
```

My Code: (main_ILOB.cpp)

ILOB > main_ILOB.cpp > main()

```cpp
#include <iostream>
#include <string>
#include "playlist_ILOB.h"

int main() {

    PlaylistNode<std::string>* playlist = nullptr;
    PlaylistNode<std::string>* current = nullptr;

    int choice;
    std::string song;

    do {
```

```
14
15          std::cout << "\n========== SONG PLAYLIST MENU ==========" << std::endl;
16          std::cout << "----------------------------------------" << std::endl;
17          std::cout << "(1) Create Playlist" << std::endl;
18          std::cout << "(2) Add Song" << std::endl;
19          std::cout << "(3) Remove Song" << std::endl;
20          std::cout << "(4) Play All Songs" << std::endl;
21          std::cout << "(5) Next Song" << std::endl;
22          std::cout << "(6) Previous Song" << std::endl;
23          std::cout << "(7) Exit" << std::endl;
24          std::cout << "Select an option: ";
25          std::cin >> choice;
26          std::cin.ignore();
27
28          switch (choice) {
29
30              // Create playlist using multiple songs
31              case 1: {
32                  std::string songs[] = {
33                      "Rosas - Yuri Dope, Skusta Clee",
34                      "Sandalan - Piolo Pascual",
35                      "Ngiti - The Juans",
36                      "Kurba - Maki"
37                  };
38
39                  int size = sizeof(songs) / sizeof(songs[0]);
40
41                  playlist = createPlaylist(songs, size);
42                  current = playlist;
43                  std::cout << "\nPlaylist created successfully." << std::endl;
44                  break;
```

```
45              }
46
47              // Add a song
48              case 2: {
49                  std::cout << "\nEnter song title: ";
50                  std::getline(std::cin, song);
51
52                  addSong(song, &playlist);
53
54                  if (current == nullptr) {
55                      current = playlist;
56                  }
57                  std::cout << "Song added successfully." << std::endl;
58                  break;
59              }
60
```

```
61              // Remove a song
62              case 3: {
63                  std::cout << "\nEnter song title to remove: ";
64                  std::getline(std::cin, song);
65
66                  if (current != nullptr && current->data == song) {
67                      current = current->next;
68                  }
69
70                  removeSong(song, &playlist);
71
72                  if (playlist == nullptr) {
73                      current = nullptr;
74                  }
75                  break;
76              }
77
78              // Play all songs
79              case 4: {
80                  std::cout << "\nPlaying all songs..." << std::endl;
81                  playAllSongs(playlist);
82                  break;
83              }
84
85              // Next song
86              case 5: {
87                  std::cout << std::endl;
88                  nextSong(current);
89                  break;
90              }
```

```
91
92              // Previous song
93              case 6: {
94                  std::cout << std::endl;
95                  previousSong(playlist, current);
96                  break;
97              }
98
99              // Exit
100             case 7:
101                 std::cout << "\nExiting playlist...\n" << std::endl;
102                 break;
103             default:
104                 std::cout << "\nInvalid option. Please try again." << std::endl;
```

```
105         }
106
107     } while (choice != 7);
108
109     return 0;
110 }
```

My Output:

Part 1:

```
========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 1

Playlist created successfully.

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 4

Playing all songs...
Rosas - Yuri Dope, Skusta Clee
Sandalan - Piolo Pascual
Ngiti - The Juans
Kurba - Maki

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
```

Part 2:

```
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 2

Enter song title: Tahanan - Adie
Song added successfully.

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 4

Playing all songs...
Rosas - Yuri Dope, Skusta Clee
Sandalan - Piolo Pascual
Ngiti - The Juans
Kurba - Maki
Tahanan - Adie

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 5
```

Part 3:

```
Now Playing: Sandalan - Piolo Pascual

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 5

Now Playing: Ngiti - The Juans

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 5

Now Playing: Kurba - Maki

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 5
```

Part 4:

```
Now Playing: Tahanan - Adie

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 5

Now Playing: Rosas - Yuri Dope, Skusta Clee

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 6

Now Playing: Tahanan - Adie

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 3

Enter song title to remove: Ngiti - The Juans
```

Part 5:

```
========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 4

Playing all songs...
Rosas - Yuri Dope, Skusta Clee
Sandalan - Piolo Pascual
Kurba - Maki
Tahanan - Adie

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 3

Enter song title to remove: Unknown Song
Song not found.
```

Part 6:

```
========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 3

Enter song title to remove: Rosas - Yuri Dope, Skusta Clee

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 4

Playing all songs...
Sandalan - Piolo Pascual
Kurba - Maki
Tahanan - Adie

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 3
```

Part 7:

```
Enter song title to remove: Sandalan - Piolo Pascual

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 3

Enter song title to remove: Kurba - Maki

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
```

Part 8:

```
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 3

Enter song title to remove: Tahanan - Adie

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 4
```

Part 9:

```
Playing all songs...
The playlist is empty.

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 5

The playlist is empty.

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 6
```

Part 10:

```
The playlist is empty.

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 9

Invalid option. Please try again.

========== SONG PLAYLIST MENU ==========
----------------------------------------
(1) Create Playlist
(2) Add Song
(3) Remove Song
(4) Play All Songs
(5) Next Song
(6) Previous Song
(7) Exit
Select an option: 7

Exiting playlist...
```

My Analysis:

The code shows how a circular linked list can be used to create and manage a song playlist where each song is connected to the next song and the last song connects back to the first song. The important code in ***createPlaylist()*** is ***tail->next = head;***, which makes the playlist circular and allows the songs to continue from the last song back to the first. The output first shows ***"Playlist created successfully."***, and when ***Play All Songs*** is selected, the four starting songs ***"Rosas - Yuri Dope, Skusta Clee," "Sandalan - Piolo Pascual," "Ngiti - The Juans,"*** and ***"Kurba - Maki"*** are printed in order. The ***addSong()*** function then adds ***"Tahanan - Adie"*** to the end of the playlist using the important code ***tail->next = newNode;*** and ***newNode->next = *head;***, which keeps the new song connected to the first song. The ***playAllSongs()*** function uses a ***do-while*** loop with while ***(current != head)*** so every song is printed once without making an endless loop even though the list is circular. The ***nextSong()*** function uses ***current = current->next;*** to move to the next song, and the output shows ***"Now Playing:"*** followed by the next song. When the last song ***"Tahanan - Adie"*** is reached and ***Next Song*** is selected again, the output returns to ***"Rosas - Yuri Dope, Skusta Clee,"*** showing that the circular connection is working properly. The ***previousSong()*** function finds the node before the current song using ***previous->next != current***, allowing the playlist to move backward even though each node only has a next pointer.

For deleting a song, ***removeSong()*** searches for the selected song, and the important code ***previous->next = current->next;*** reconnects the remaining songs after the selected node is removed. The output also shows ***"Song not found."*** when ***"Unknown Song"*** is entered, showing that the program checks if the song exists before deleting it. When the songs are removed one by one, the code also handles the special case where only one song remains and sets ****head = nullptr;*** after the last song is deleted. The output then shows ***"The playlist is empty."*** when ***Play All Songs***, ***Next Song***, or ***Previous Song*** is selected while there are no songs left. The program also handles an invalid menu choice through the default case, where entering an option outside 1 to 7 displays ***"Invalid option. Please try again."*** and returns to the menu. Finally, the output shows that the important playlist operations are working properly, including creating the playlist, adding and removing unique songs, playing all songs, moving to the next and previous songs, handling a song that does not exist, and handling an empty playlist. Moreover, it also shows why a circular linked list is useful for a music playlist because the songs can continue in a loop instead of stopping after the last song.

Conclusion:

The activity helped show how linked lists work in different ways in C++ and how nodes can be connected and managed using pointers. From the procedures, the singly linked list helped in understanding how the next pointer connects one node to another, while the doubly linked list added the prev pointer so the list can be accessed in both forward and backward directions. The different operations such as traversal, insertion at the head, insertion at any part, insertion at the end, deletion of a node, and deletion of the whole list made the pointer connections more understandable. The supplementary activity about the song playlist made the lesson more practical because the circular linked list allowed the songs to continue from the last song back to the first song. The use of important code like tail->next = head;, current = current->next;, and previous->next = current->next; showed how the circular playlist can move, add, and remove songs properly.

Some difficulties were faced while understanding the pointer connections, dynamic memory, and the changes needed when inserting or deleting nodes. The doubly linked list was also a little more difficult because both next and prev needed to be updated correctly, and one missed connection can make the list work incorrectly. The circular linked list also needed more attention because it does not end with nullptr, so the traversal must know when it reaches the starting node again. Overall, the activity was done fairly well because the required linked list operations were successfully implemented and the outputs showed the expected results. There is still some area for improvement, especially in understanding pointers faster, managing dynamic memory, and handling special cases like an empty list, deleting the head, deleting the last node, and moving around a circular list. More practice in creating linked list programs without depending too much on examples would help make the process more familiar and improve confidence in writing, debugging, and checking the code.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT, Grammarly**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **formatting, explanation of concepts, grammar correction**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **The AI was used to explain how the code works by giving a detailed explanation on the different parts of the code. Moreover, the AI was used to assist in improving the grammar of the code analysis, allowing the explanation to be written more clearly and fluently in English.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.